

Shell Programming -Functions

Recall the Purposes of Functions

Like many other programming languages, a function is a collection of statements that execute a specified task. A function can be used to:

- assist with code reuse;
- enhance the program's readability;
- modularize the software: break down a complicated procedure into simpler subroutines that can subsequently be used to accomplish the more complex routine;
- allow for easy maintenance.

Some Functions Principles in Shell Programming

General Syntax:

```
function_name() {  
    statement1  
    statement2  
    . . . .  
}
```

- A function must be defined before the function call.
- Use `local` keyword to declare a local variable (within a function scope).
- These parameter variables `$*`, `$@`, `$#`, `$0`, `$1`, `$2`, `...` can be used to hold argument values passed in command-line when the function is called.
- A function can return a value, with the `return` keyword. However, it can only return a number within `[0, 255]`. (Try this out using the example in the next slide).

Function Example - Local Keyword

```
#!/bin/sh
#ex.sh
studentId()
{
    local id
    echo -n "what's your student id "
    read id
    echo "my id is $id"
}
studentId
exit 0
```

Output after running ./ex.sh

```
what's your student id 111
my id is 111
```

Global Variable vs. Local Variable

- By default, every variable we create in Bash is global. They can be used in any function, loop, or conditional statement within the script.
- Unlike global variables, local variables are only visible within the block of code or function in which they are declared. Outside of that block, the variable is not recognized.

```
#!/bin/sh
function foo() {
    x=3
    local y=5
}
foo
echo "x=$x"
echo "y=$y"
```

Output:

```
x=3
y=
```

Function With Return Values Example

```
#!/bin/sh
find_avg() {
    len=$#
    sum=0
    for x in "$@"
    do
        sum=$((sum + x))
    done
    avg=$((sum/len))
    return $avg
}
find_avg 30 40 50 60
printf "%f" "$?"
printf "\n"
exit 0
```

Output:

45.000000

```
#!/bin/sh
find_avg() {
    len=$#
    sum=0
    for x in "$@"
    do
        sum=$((sum + x))
    done
    avg=$((sum/len))
    return $avg
}
find_avg 300 400 500 600
printf "%f" "$?"
printf "\n"
exit 0
```

Output:

194.000000 # error number

Expr Command in Arithmetic Expressions

The `expr` command can be used to evaluate arithmetic or comparison expressions. There must be a space around the arithmetic or comparison operator for the `expr` command to work properly.

```
add() {  
    local rval=$(expr $1 + $2) # same as: local rval=`expr $1 + $2`  
    return $rval  
}  
subtract() {  
    local rval=`expr $1 - $2`  
    return $rval  
}  
add 2 3  
a=$?      # $? Hold the resulting value  
echo $a  
  
subtract 4 1  
echo $?  
  
exit 0
```

Output:

5
3

Expr Command in Arithmetic Expressions (Cont')

```
#!/bin/sh
multiply(){
    local rval=`expr $1 \* $2`
    return $rval
}
divide() {
    local rval=`expr $1 / $2` # integer division
    return $rval
}

multiply 2 3
echo $?

divide 5 4
echo $?
exit 0
```

Output:

6
1

Expr Command in Comparison Expressions

Watch out space around = operators: sometimes there must be no space, sometimes there must be a space.

```
#!/bin/sh
x=3
y=5
b1=`expr $x = $y`
b2=`expr $x \< $y`
b3=`expr $x \> $y`
b4=`expr $x \<= $y`
b5=`expr $x \>= $y`
b6=`expr $x \!= $y`
echo "b1=$b1, b2=$b2, b3=$b3, b4=$b4, b5=$b5, b6=$b6"
exit 0
```

Output:

```
b1=0, b2=1, b3=0, b4=1, b5=0, b6=1
```

String Operations

- to get length of a string

```
x=geeks  
echo "${#x}"
```

- string concatenation

```
x=geeks  
x="$x $x"  
echo $x
```

- extract a substring from a string

```
x="how are you"  
echo ${x:3} #return a substring starting at index 3 to the end  
echo ${x:3:2} # return a substring starting at index 3, of length 2
```

```
output:  
are you  
a
```

printf

`printf` command is similar to `echo`, but preferred for generating formatted output.

Syntax:

```
printf "format specification" parameter1 parameter2 ...
```

All characters in the format specification other than `\%` and `\` appear literally in the output.

Example:

```
printf "%d %s %c \n" 2 "Hi" 'h'    # output: 2 Hi h
printf "%d, %s, %c \n" 2 "Hi" 'h'    # output: 2, Hi, h
printf "%d, \"%s\", %c \n" 2 "Hi" 'h'  # output: 2, "Hi", h
printf "%f\n" 3.0    # output: 3.00000
printf "%d\n " 0xF      #Convert a hex number to decimal
printf "0x%X\n " 15     #Convert a decimal number to Hex
```

Can you infer the meanings of those specifiers: `%d`, `%s`, `%f`, `%c`?

export Command

The `export` command is used to export variables to child processes. For example,

```
#!/bin/sh
# export.sh: export variables to child scripts
export a="Cool Cool"
export b="Fun Fun"
./import1.sh
./import2.sh
exit 0
```

```
#!/bin/sh
# import1.sh
echo "$a"
exit 0
```

```
#!/bin/sh
# import2.sh
echo "$b"
exit 0
```

```
./export.sh
```